



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)

Кафедра прикладной математики (ПМ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №5

по дисциплине «Технологии и инструментарий машинного обучения»

Студент группы *ИНБО-20-23. Школьный И.В.*

(подпись)

Преподаватель *Трушин С.М.*

(подпись)

Москва 2025 г.

СОДЕРЖАНИЕ

Введение.....	3
1 Метод решения	4
1.1 Задание 1. Дерево решений.....	6
1.2 Задание 2. Случайный лес	10
1.3 Задание 3. Градиентный бустинг.....	12
1.4 Задание 4. Стэкинг	15
1.5 Сравнение всех моделей.....	16
Вывод.....	17

ВВЕДЕНИЕ

Цель практической работы №5 — изучить принципы построения и работы деревьев решений, освоить методы ансамблирования (bagging, boosting, stacking) для решения задач классификации.

1 МЕТОД РЕШЕНИЯ

Необходимо подобрать датасет, в котором не менее 1000 строк, хотя бы 3 числовых признака и два категориальных. Импорт представлен в Листинге 1.

Листинг 1 — Импорт библиотек и датасета

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

df = pd.read_csv('loan_data.csv')
print(f'Размер датасета: {df.shape}')
df.head()
```

Так выглядят первые 5 строк датасета (Таблица 1).

Таблица 1 — Первые 5 строк датасета

person_age	person_gender	person_education	person_income	person_employment	person_home_ownership	loan_amount	loan_intent	loan_int_rate	loan_percent_income	cb_person_cred_hist_length	credit_score	previous_loan_defaults_on_file	loan_status
22.0	female	Master	71948.0	0	RENT	35000.0	PERSONAL	16.02	0.49	3.0	561	No	1
21.0	female	High School	12282.0	0	OWN	1000.0	EDUCATION	11.14	0.08	2.0	504	Yes	0
25.0	female	High School	12438.0	3	MORTGAGE	5500.0	MEDICAL	12.87	0.44	3.0	635	No	1
23.0	female	Bachelor	79753.0	0	RENT	35000.0	MEDICAL	15.23	0.44	2.0	675	No	1
24.0	male	Master	66135.0	1	RENT	35000.0	MEDICAL	14.27	0.53	4.0	586	No	1

Проведем разведочный анализ в `logitom`. Откроем эксель файл через узел,

применим узел дубликатов и противоречий и отфильтруем строки по полю «дубликат» и «противоречие» (Рисунок 1).

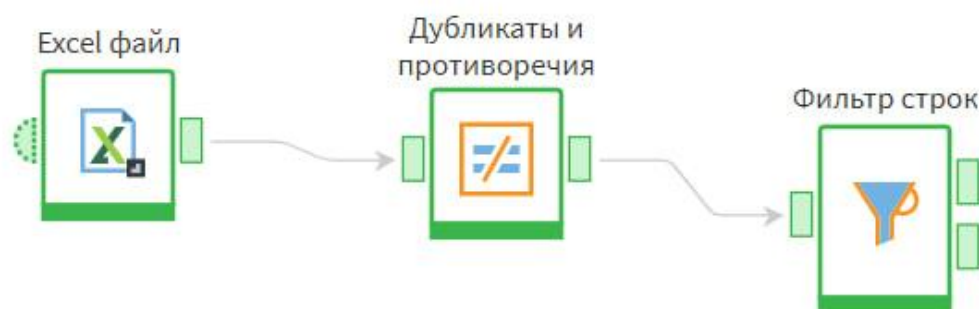


Рисунок 1 — Сценарий поиска дубликатов и противоречий

Перейдем в результат выполнения сценария (Рисунок 2).

Фильтр строк • Быстрый просмотр

Соответствуют условию

Не соответствуют условию

#	01 Дубликат	12 Группа дубликата	01 Противореч...	12 Группа противореч...	90 person_a...	12 person_gender	12 p
---	-------------	---------------------	------------------	-------------------------	----------------	------------------	------

Таблица

Форма

Рисунок 2 — Строки-дубликаты или противоречия

Ни одна из 45000 строк не была отнесена к дубликатам, значит, мы работаем с чистым датасетом.

Выполним масштабирование числовых признаков. Это процесс преобразования числовых данных так, чтобы они находились в общем сопоставимом масштабе. Для этого воспользуемся функцией стандартизации `StandardScaler`, который хорош для большинства случаев. Код программы представлен в Листинге 2.

Листинг 2 — Масштабирование числовых признаков

```
numeric_cols = ['person_age', 'person_income', 'person_emp_exp', 'loan_amnt',  
'loan_int_rate', 'loan_percent_income', 'cb_person_cred_hist_length',  
'credit_score']  
scaler = StandardScaler()  
df[numeric_cols] = scaler.fit_transform(df[numeric_cols])
```

Далее выполним кодирование категориальных признаков так же, как и в прошлой работе (Листинг 3).

Листинг 3 — Кодирование категориальных признаков

```
categorical_cols = ['person_gender', 'person_education',  
'person_home_ownership', 'loan_intent', 'previous_loan_defaults_on_file']  
label_encoders = {}  
for col in categorical_cols:  
    le = LabelEncoder()  
    df[col] = le.fit_transform(df[col])  
    label_encoders[col] = le
```

После этого экспортируем файл, чтобы провести параллельную работу на `loginom`.

1.1 Задание 1. Дерево решений

Дерево решений — это модель машинного обучения, которая строит иерархическую структуру правил вида «Если признак $A < X$, то...». Основными компонентами являются корень, узлы и листья.

Выполним построение дерева решений и подберем лучшие параметры (Листинг 4).

Листинг 4 — Дерево решений

```
tree_gini = DecisionTreeClassifier(criterion='gini', random_state=42)
tree_gini.fit(X_train, y_train)
tree_entropy = DecisionTreeClassifier(criterion='entropy', random_state=42)
tree_entropy.fit(X_train, y_train)

param_grid = {
    'max_depth': [3, 5, 10, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'criterion': ['gini', 'entropy']
}
grid_search = GridSearchCV(DecisionTreeClassifier(random_state=42), param_grid,
cv=5, scoring='f1')
grid_search.fit(X_train, y_train)
best_tree = grid_search.best_estimator_
print(f"Лучшие параметры: {grid_search.best_params_}")
```

Получились следующие значения:

- accuracy: 0.918;
- precision: 0.882;
- recall: 0.730;
- f1-Score: 0.799.

Дерево достигло глубины 10, критерий entropy показал лучшие результаты, чем gini. Модель консервативна, чаще правильно предсказывает одобрение, но пропускает часть положительных случаев.

Рос-кривая выглядит следующим образом (Рисунок 3).

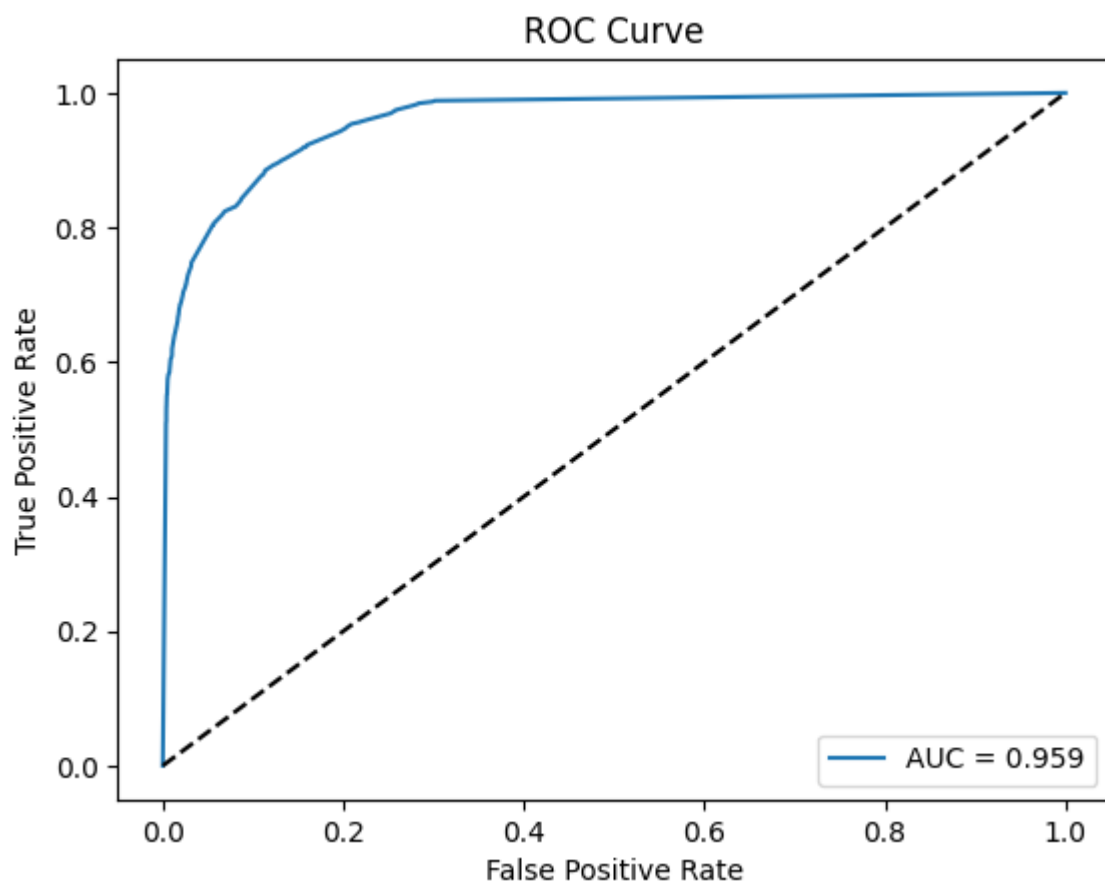


Рисунок 3 — ROC-кривая для дерева решений

На Рисунке 4 изображена кривая обучения.

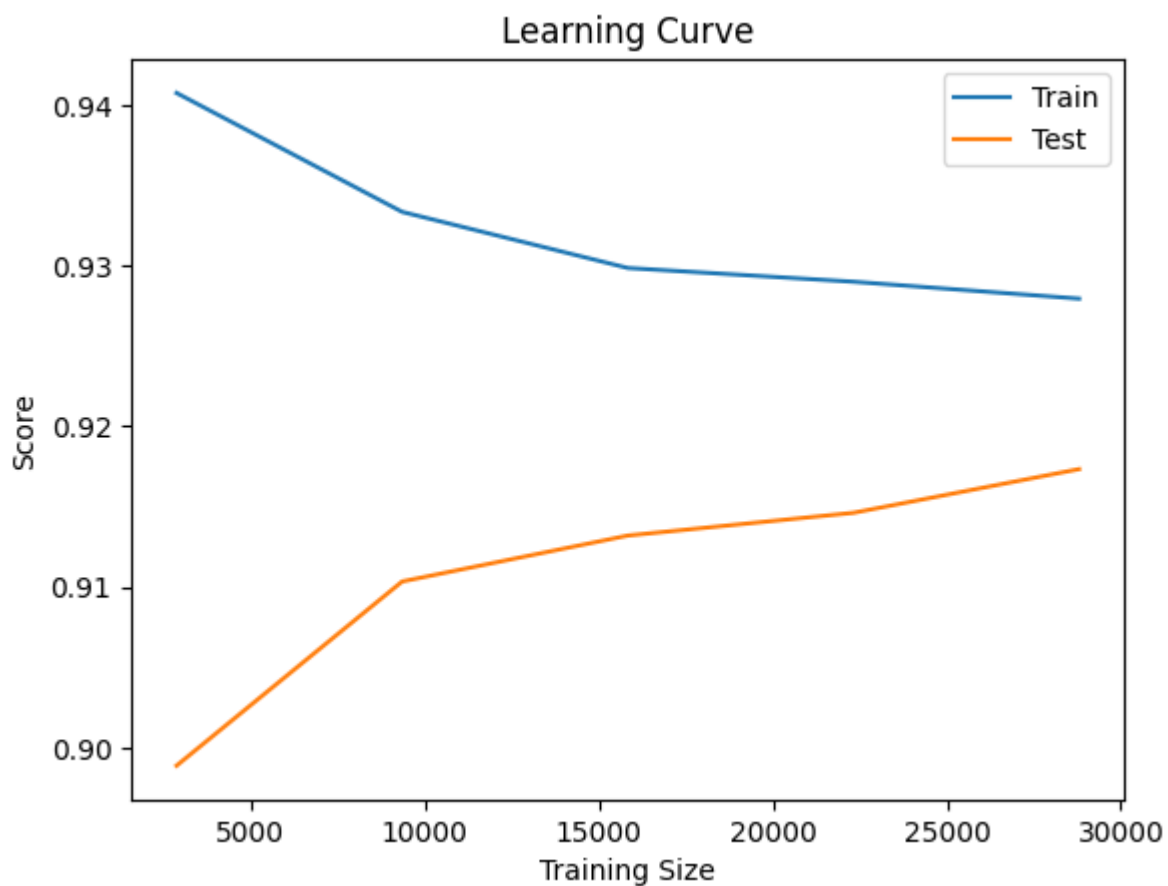


Рисунок 4 — Кривая обучения для дерева решений

Посмотрим на визуализацию дерева решений (Рисунок 5).

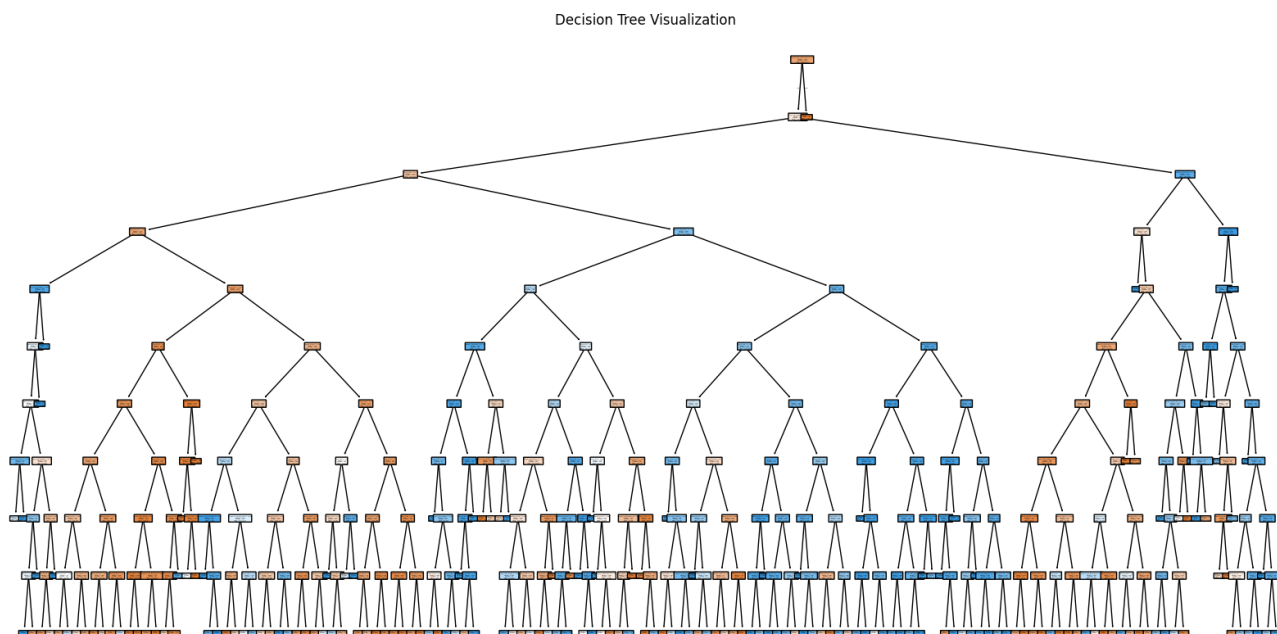


Рисунок 5 — дерево решений

Дерево решений было успешно построено и проанализировно.

1.2 Задание 2. Случайный лес

Random Forest — ансамбль из множества деревьев, обученных на случайных подвыборках с использованием случайных подмножеств признаков.

Основные преимущества:

1. Снижение дисперсии за счёт усреднения.
2. Устойчивость к переобучению.
3. Оценка важности признаков.

Реализуем метод случайного леса программно (Листинг 5).

Листинг 5 — Случайный лес

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, RandomizedSearchCV

X_small, _, y_small, _ = train_test_split(
    X_train, y_train, train_size=0.3, stratify=y_train, random_state=42
)
param_dist = {
    'n_estimators': [50, 100, 150],
    'max_depth': [5, 10, 15, None],
    'min_samples_split': [10, 20, 30],
    'min_samples_leaf': [1, 5, 10],
}
random_search = RandomizedSearchCV(
    RandomForestClassifier(n_jobs=-1, random_state=42),
    param_dist,
    n_iter=20,
    cv=3,
    scoring='f1',
    n_jobs=-1,
    random_state=42,
    verbose=2
)
random_search.fit(X_small, y_small)
best_rf = random_search.best_estimator_
print(f"Лучшие параметры: {random_search.best_params_}")
best_rf.fit(X_train, y_train)
evaluate_model(best_rf, X_test, y_test)
```

Получаем следующую статистику:

- accuracy: 0.927;
- precision: 0.888;
- recall: 0.771;
- f1-Score: 0.825.

ROC-кривая изображена на Рисунке 6.

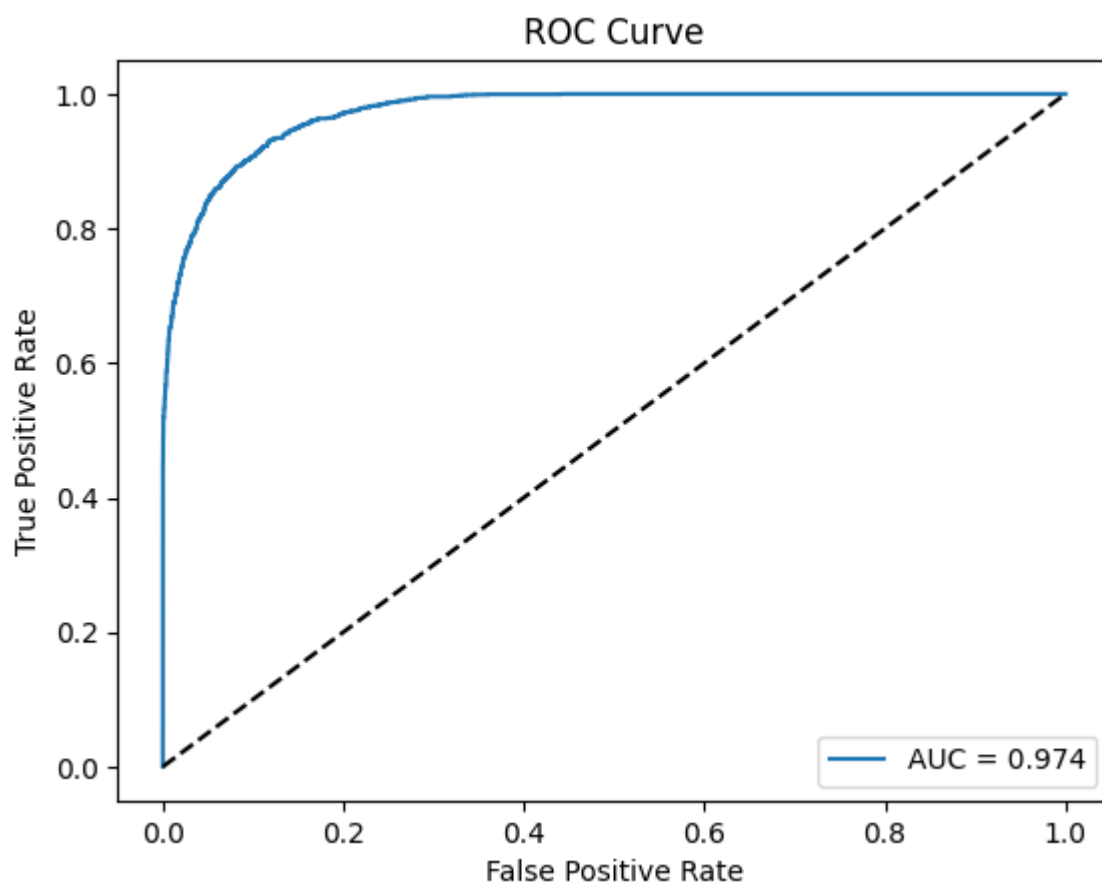


Рисунок 6 — ROC-кривая для случайного леса

Посмотрим на кривую обучения (Рисунок 7).

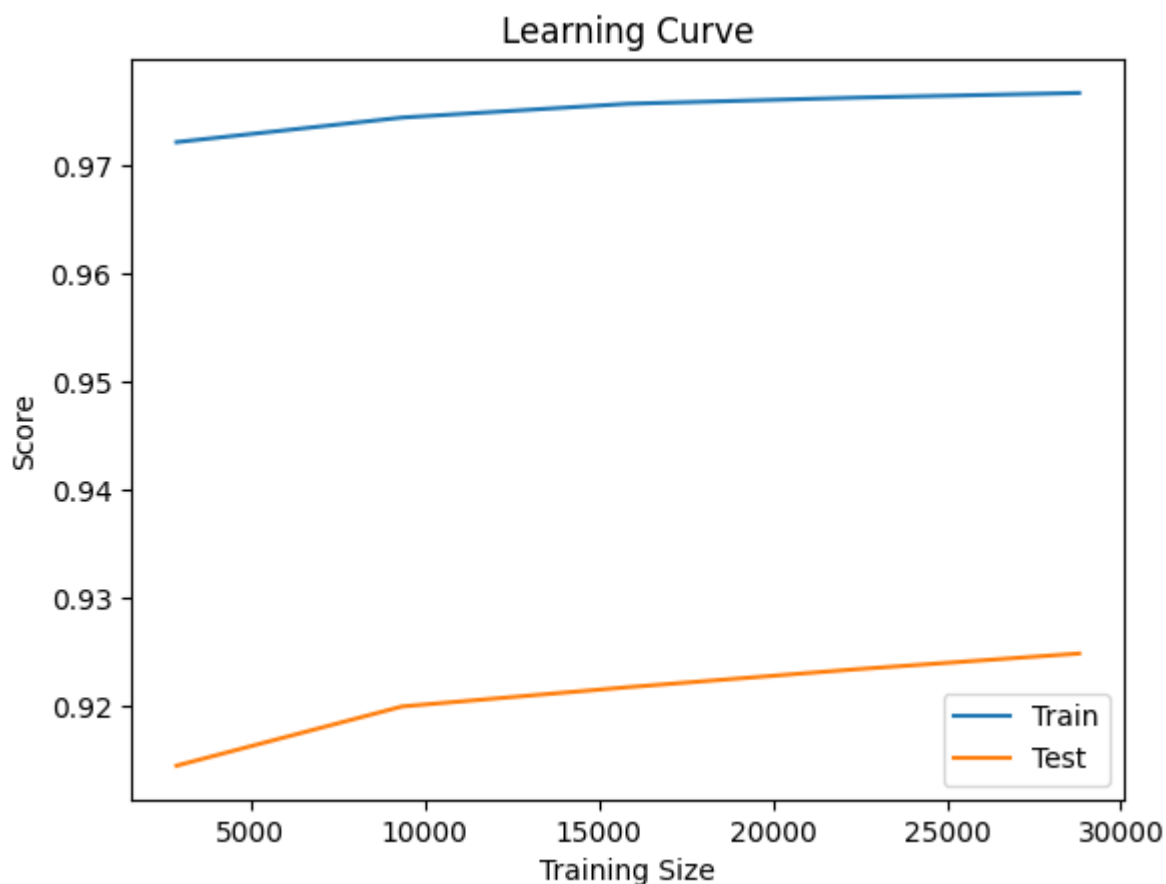


Рисунок 7 — Кривая обучения для случайного леса

Random Forest улучшил все метрики, особенно Recall, что означает лучшее выявление истинно одобренных кредитов.

1.3 Задание 3. Градиентный бустинг

Градиентный бустинг — последовательный алгоритм, где каждая следующая модель обучается на ошибках (градиентах функции потерь) предыдущих. Особенности:

1. Высокая точность за счёт снижения смещения.
2. Чувствительность к переобучению.
3. Требуется тщательной настройке гиперпараметров.

В данной работе мы будем использовать GradientBoostingClassifier из библиотеки sklearn и XGBoost (eXtreme Gradient Boosting).

Реализуем градиентный бустинг программно (Листинг 6).

Листинг 6 — Градиентный бустинг

```
from sklearn.ensemble import GradientBoostingClassifier
from xgboost import XGBClassifier
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint, uniform
import numpy as np

sample_size = int(0.3 * len(X_train))
indices = np.random.choice(len(X_train), sample_size, replace=False)
X_small = X_train.iloc[indices]
y_small = y_train.iloc[indices]
gb = GradientBoostingClassifier(random_state=42)
gb.fit(X_small, y_small)
xgb_base = XGBClassifier(random_state=42, eval_metric='logloss', n_jobs=-1)
param_dist_xgb = {
    'n_estimators': randint(50, 200),
    'learning_rate': uniform(0.01, 0.3),
    'max_depth': randint(3, 10),
    'subsample': uniform(0.7, 0.3),
    'colsample_bytree': uniform(0.7, 0.3)
}
random_search_xgb = RandomizedSearchCV(
    xgb_base,
    param_dist_xgb,
    n_iter=10,
    cv=3,
    scoring='f1',
    random_state=42,
    n_jobs=-1,
    verbose=1
)
random_search_xgb.fit(X_small, y_small)
best_xgb = random_search_xgb.best_estimator_
gb.fit(X_train, y_train)
best_xgb.fit(X_train, y_train)
print(f"Лучшие параметры XGBoost: {random_search_xgb.best_params}")
```

Посмотрим статистику:

- `n_estimators=50`: Train F1=0.8288, Test F1=0.8092;
- `n_estimators=100`: Train F1=0.8677, Test F1=0.8247;
- `n_estimators=150`: Train F1=0.8954, Test F1=0.8285;
- `n_estimators=200`: Train F1=0.9179, Test F1=0.8295;
- `n_estimators=250`: Train F1=0.9375, Test F1=0.8312;
- `n_estimators=300`: Train F1=0.9513, Test F1=0.8298.

Заметим, что при `<150` деревьев модель недообучена, а при `>250` – переобучение. График демонстрирует эту зависимость (Рисунок 8).

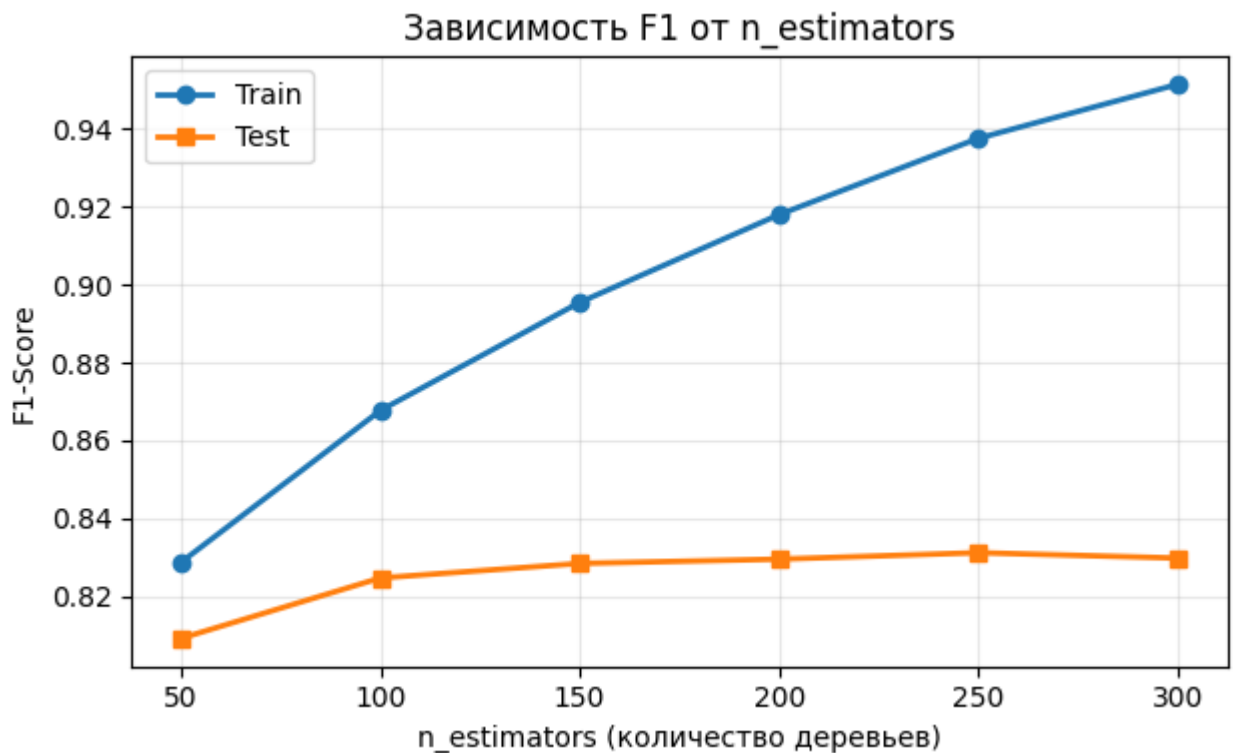


Рисунок 8 — График зависимости f1-score от количества деревьев

Посмотрим зависимость f1-score от learning_rate:

- learning_rate=0.001: F1 = 0.780;
- learning_rate=0.010: F1 = 0.810;
- learning_rate=0.050: F1 = 0.825;
- learning_rate=0.100: F1 = 0.828;
- learning_rate=0.200: F1 = 0.831;
- learning_rate=0.300: F1 = 0.828;
- learning_rate=0.500: F1 = 0.820.

Слишком маленький learning_rate требует больше итераций, но при этом слишком большой learning_rate вызывает нестабильность, поэтому оптимальное значение: 0.1-0.3.

1.4 Задание 4. Стэкинг

Стэкинг — двухуровневый ансамбль:

1. Базовые модели: разные алгоритмы (Random Forest, SVM, Logistic Regression, KNN).
2. Метамодел: обучается на предсказаниях базовых моделей.

Реализация представлена в Листинге 7.

Листинг 7 — Стэкинг

```
sample_size = min(5000, len(X_train))
indices = np.random.choice(len(X_train), sample_size, replace=False)
X_small = X_train.iloc[indices]
y_small = y_train.iloc[indices]
base_models = [
    ('rf', RandomForestClassifier(
        n_estimators=50,
        max_depth=10,
        n_jobs=-1,
        random_state=42
    )),
    ('svm', SVC(
        kernel='rbf',
        C=1.0,
        probability=True,
        random_state=42
    )),
    ('lr', LogisticRegression(
        max_iter=500,
        random_state=42
    )),
    ('knn', KNeighborsClassifier(n_neighbors=5))
]
meta_model = LogisticRegression(max_iter=1000, random_state=42)
stacking = StackingClassifier(
    estimators=base_models,
    final_estimator=meta_model,
    cv=3,
    n_jobs=-1
)
stacking.fit(X_small, y_small)
y_pred_stacking = stacking.predict(X_test)
print("Результаты Stacking")
print(f"Accuracy: {accuracy_score(y_test, y_pred_stacking):.4f}")
print(f"F1-Score: {f1_score(y_test, y_pred_stacking):.4f}")
print(f"Precision: {precision_score(y_test, y_pred_stacking):.4f}")
print(f"Recall: {recall_score(y_test, y_pred_stacking):.4f}")
```

Посмотрим на метрики:

- accuracy: 0.9156;
- f1-Score: 0.7996;

- precision: 0.8460;
- recall: 0.7580.

1.5 Сравнение всех моделей

Проведем общий анализ и выявим наиболее оптимальный метод (Таблица 2).

Таблица 2 — Сравнительный анализ

Модель	Accuracy	F1-Score	Precision	Recall
XGBoost	0.9324	0.8380	0.8968	0.7865
Random Forest	0.9274	0.8253	0.8877	0.7710
Gradient Boosting	0.9220	0.8121	0.8738	0.7585
Stacking	0.9156	0.7996	0.8460	0.7580
Decision Tree	0.9182	0.7987	0.8816	0.7300

По точности и по f1-score лучшим оказался XGBoost, значит он подходит лучше остальных.

ВЫВОД

В ходе выполнения практической работы №5 были успешно изучены принципы построения и работы деревьев решений, освоены методы ансамблирования (bagging, boosting, stacking) для решения задач классификации.